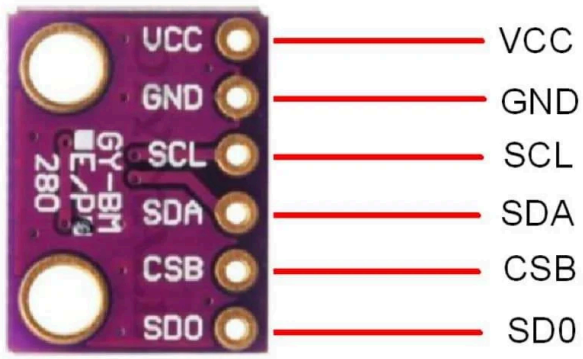
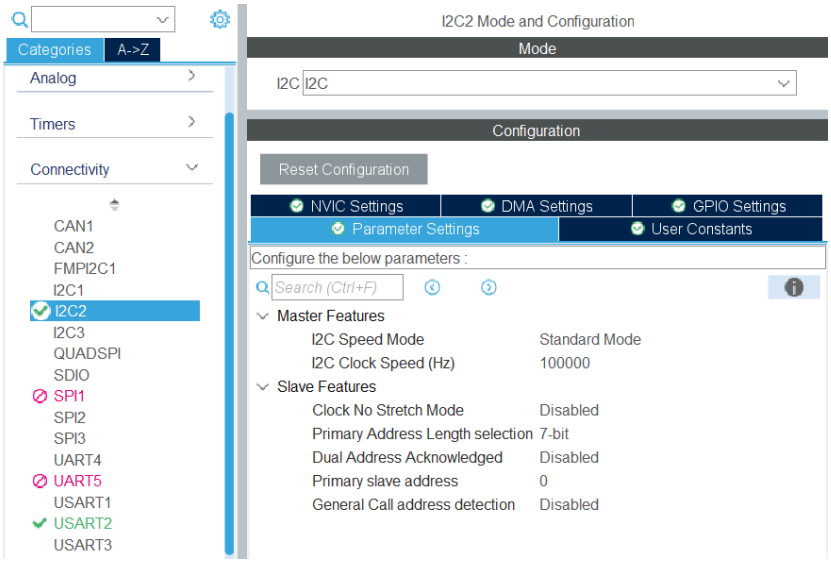
# BME280 with STM32F446RE




## Pin Configuration

1. **Enable I2C2 peripheral:**
   - Go to **Connectivity** → **I2C2**
   - Set **I2C** to **I2C**
   - The pins should now show:
     - **PB10** → I2C2_SCL (Clock Line)
     - **PC12** → I2C2_SDA (Data Line)



## USART2 Parameters

1. **Basic Parameters:**
   - **Baud Rate:** 115200 Bits/s
   - **Word Length:** 8 Bits (including Parity)
   - **Parity:** None
   - **Stop Bits:** 1
   - **Data Direction:** Receive and Transmit
   - **Over Sampling:** 16 Samples

## Connections

- **BMP280 VCC** → **3.3V** (Pin 3.3V)
- **BMP280 GND** → **GND** (Any GND pin)
- **BMP280 SCL** → **PB10** (I2C2_SCL)
- **BMP280 SDA** → **PC12** (I2C2_SDA)

## BMP280 I2C Address

- Default I2C address: **0x76** (when SDO pin is connected to GND)
- Alternative address: **0x77** (when SDO pin is connected to VCC)
- In the code, address is left-shifted: `0x76 << 1 = 0xEC`

## CODE

```
/* USER CODE END Header */
/* Includes
------------------------------------------------------------------*/
#include "main.h"
#include <stdio.h>
#include <math.h>
/* Private includes
----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef
-----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define
------------------------------------------------------------*/
/* USER CODE BEGIN PD */
```

```
#define BMP280_dev_address 0x76<<1  // BMP280 I2C address (shifted for HAL)
/* USER CODE END PD */
/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables ---------------------------------------------------------*/
I2C_HandleTypeDef hi2c2;
UART_HandleTypeDef huart2;
/* USER CODE BEGIN PV */
// Calibration parameters
uint16_t Dig_T1, Dig_P1;
int16_t Dig_T2, Dig_T3, Dig_P2, Dig_P3, Dig_P4, Dig_P5, Dig_P6, Dig_P7, Dig_P8, Dig_P9;
// Raw and processed data
uint32_t temperature_raw, pressure_raw;
float temperature, pressure, altitude;
char uart_buffer[100];
/* USER CODE END PV */
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C2_Init(void);
static void MX_USART2_UART_Init(void);
/* USER CODE BEGIN PFP */
void BMP280_init(void);
void BMP280_get_calib_values(void);
void BMP280_calc_values(void);
uint8_t I2C_Read_Register(uint8_t device_address, uint8_t register_address);
void I2C_Write_Register(uint8_t device_address, uint8_t register_address, uint8_t data);
/* USER CODE END PFP */
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
/**
 * @brief  Read a register from I2C device
 * @param  device_address: I2C device address
 * @param  register_address: Register address to read
```

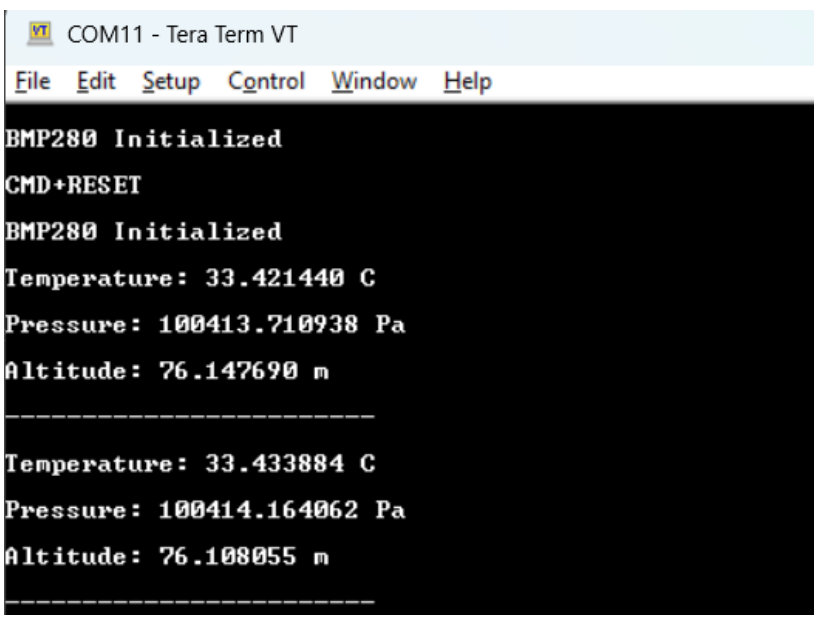
```
 * @retval Register value
 */
uint8_t I2C_Read_Register(uint8_t device_address, uint8_t
register_address)
{
   uint8_t data;
   HAL_I2C_Master_Transmit(&hi2c2, device_address, &register_address, 1,
HAL_MAX_DELAY);
   HAL_I2C_Master_Receive(&hi2c2, device_address, &data, 1,
HAL_MAX_DELAY);
   return data;
}
/**
 * @brief  Write to a register of I2C device
 * @param  device_address: I2C device address
 * @param  register_address: Register address to write
 * @param  data: Data to write
 * @retval None
 */
void I2C_Write_Register(uint8_t device_address, uint8_t
register_address, uint8_t data)
{
   uint8_t buffer[2];
   buffer[0] = register_address;
   buffer[1] = data;
   HAL_I2C_Master_Transmit(&hi2c2, device_address, buffer, 2,
HAL_MAX_DELAY);
}
/**
 * @brief  Initialize BMP280 sensor
 * @param  None
 * @retval None
 */
void BMP280_init(void)
{
   // Configure BMP280 registers
   // Register 0xF5: standby time 500ms (100), filter 16 (100), SPI
disabled (00)
   I2C_Write_Register(BMP280_dev_address, 0xF5, 0b10010000);
   // Register 0xF4: osrs_t x2 (010), osrs_p x16 (101), mode normal (11)
   I2C_Write_Register(BMP280_dev_address, 0xF4, 0b01010111);
   // Get calibration values from sensor
   BMP280_get_calib_values();
```

```
}
/**
 * @brief  Get calibration values from BMP280
 * @param  None
 * @retval None
 */
void BMP280_get_calib_values(void)
{
   uint8_t rx_buff[24], starting_address = 0x88;
   // Read 24 bytes of calibration data starting from address 0x88
   HAL_I2C_Master_Transmit(&hi2c2, BMP280_dev_address, &starting_address,
1, HAL_MAX_DELAY);
   HAL_I2C_Master_Receive(&hi2c2, BMP280_dev_address, &rx_buff[0], 24,
HAL_MAX_DELAY);
   // Parse calibration data
   Dig_T1 = (rx_buff[0]) + (rx_buff[1] << 8);
   Dig_T2 = (rx_buff[2]) + (rx_buff[3] << 8);
   Dig_T3 = (rx_buff[4]) + (rx_buff[5] << 8);
   Dig_P1 = (rx_buff[6]) + (rx_buff[7] << 8);
   Dig_P2 = (rx_buff[8]) + (rx_buff[9] << 8);
   Dig_P3 = (rx_buff[10]) + (rx_buff[11] << 8);
   Dig_P4 = (rx_buff[12]) + (rx_buff[13] << 8);
   Dig_P5 = (rx_buff[14]) + (rx_buff[15] << 8);
   Dig_P6 = (rx_buff[16]) + (rx_buff[17] << 8);
   Dig_P7 = (rx_buff[18]) + (rx_buff[19] << 8);
   Dig_P8 = (rx_buff[20]) + (rx_buff[21] << 8);
   Dig_P9 = (rx_buff[22]) + (rx_buff[23] << 8);
}
/**
 * @brief  Calculate temperature and pressure values
 * @param  None
 * @retval None
 */
void BMP280_calc_values(void)
{
   uint8_t status, rx_buff[6], starting_address = 0xF7;
   // Wait for measurement to complete
   do {
       status = I2C_Read_Register(BMP280_dev_address, 0xF3);
   } while(((status & 0b00001000) == 8) || ((status & 0b00000001) == 1));
   // Read 6 bytes of pressure and temperature data
```

```
    HAL_I2C_Master_Transmit(&hi2c2, BMP280_dev_address, &starting_address, 1, HAL_MAX_DELAY);
    HAL_I2C_Master_Receive(&hi2c2, BMP280_dev_address, &rx_buff[0], 6, HAL_MAX_DELAY);
    // Parse raw temperature data
    volatile uint32_t temp[3];
    temp[2] = rx_buff[3];
    temp[1] = rx_buff[4];
    temp[0] = rx_buff[5];
    temperature_raw = (temp[2] << 12) + (temp[1] << 4) + (temp[0] >> 4);
    // Parse raw pressure data
    temp[2] = rx_buff[0];
    temp[1] = rx_buff[1];
    temp[0] = rx_buff[2];
    pressure_raw = (temp[2] << 12) + (temp[1] << 4) + (temp[0] >> 4);
    // Temperature compensation
    double var1, var2;
    var1 = (((double)temperature_raw) / 16384.0 - ((double)Dig_T1) / 1024.0) * ((double)Dig_T2);
    var2 = ((((double)temperature_raw) / 131072.0 - ((double)Dig_T1) / 8192.0) *
            (((double)temperature_raw) / 131072.0 - ((double)Dig_T1) / 8192.0)) * ((double)Dig_T3);
    double t_fine = (int32_t)(var1 + var2);
    volatile float T = (var1 + var2) / 5120.0;
    temperature = T;
    // Pressure compensation
    var1 = ((double)t_fine / 2.0) - 64000.0;
    var2 = var1 * var1 * ((double)Dig_P6) / 32768.0;
    var2 = var2 + var1 * ((double)Dig_P5) * 2.0;
    var2 = (var2 / 4.0) + (((double)Dig_P4) * 65536.0);
    var1 = (((double)Dig_P3) * var1 * var1 / 524288.0 + ((double)Dig_P2) * var1) / 524288.0;
    var1 = (1.0 + var1 / 32768.0) * ((double)Dig_P1);
    if (var1 == 0) {
        pressure = 0; // Avoid division by zero
    } else {
        volatile double p = 1048576.0 - (double)pressure_raw;
        p = (p - (var2 / 4096.0)) * 6250.0 / var1;
        var1 = ((double)Dig_P9) * p * p / 2147483648.0;
        var2 = p * ((double)Dig_P8) / 32768.0;
        p = p + (var1 + var2 + ((double)Dig_P7)) / 16.0;
        pressure = p;
```

```
    }
    // Calculate altitude
    altitude = 44330.0f * (1 - powf(pressure / 101325.0f, 1.0f / 5.255f));
}
/* USER CODE END 0 */
/**
 * @brief  The application entry point.
 * @retval int
 */
int main(void)
{
 /* USER CODE BEGIN 1 */
 /* USER CODE END 1 */
 /* MCU Configuration--------------------------------------------------------*/
 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
 HAL_Init();
 /* USER CODE BEGIN Init */
 /* USER CODE END Init */
 /* Configure the system clock */
 SystemClock_Config();
 /* USER CODE BEGIN SysInit */
 /* USER CODE END SysInit */
 /* Initialize all configured peripherals */
 MX_GPIO_Init();
 MX_I2C2_Init();
 MX_USART2_UART_Init();
 /* USER CODE BEGIN 2 */
 // Initialize BMP280 sensor
 HAL_Delay(1000); // Wait for sensor to stabilize
 BMP280_init();
 sprintf(uart_buffer, "BMP280 Initialized\r\n");
 HAL_UART_Transmit(&huart2, (uint8_t*)uart_buffer, strlen(uart_buffer), HAL_MAX_DELAY);
 /* USER CODE END 2 */
 /* Infinite loop */
 /* USER CODE BEGIN WHILE */
 while (1)
 {
   /* USER CODE END WHILE */
   /* USER CODE BEGIN 3 */
```

```
    // Calculate temperature and pressure
    BMP280_calc_values();
    // Send data via UART
    sprintf(uart_buffer, "Temperature: %.2f C\r\n", temperature);
    HAL_UART_Transmit(&huart2, (uint8_t*)uart_buffer, strlen(uart_buffer),
HAL_MAX_DELAY);
    sprintf(uart_buffer, "Pressure: %.2f Pa\r\n", pressure);
    HAL_UART_Transmit(&huart2, (uint8_t*)uart_buffer, strlen(uart_buffer),
HAL_MAX_DELAY);
    sprintf(uart_buffer, "Altitude: %.2f m\r\n", altitude);
    HAL_UART_Transmit(&huart2, (uint8_t*)uart_buffer, strlen(uart_buffer),
HAL_MAX_DELAY);
    sprintf(uart_buffer, "------------------------\r\n");
    HAL_UART_Transmit(&huart2, (uint8_t*)uart_buffer, strlen(uart_buffer),
HAL_MAX_DELAY);
    HAL_Delay(2000); // Wait 2 seconds before next reading
  }
  /* USER CODE END 3 */
}
```

## OUTPUT

Open Teraterm at a baud rate of 115200 and you should be able to see the output.